

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DANIELA ARALDI

**GERAÇÃO DE REGRAS *SIGMA* POR MEIO DE AGENTE AUTÔNOMO COM
RAG: ANÁLISE COMPARATIVA DE ESTRATÉGIAS**

PATO BRANCO

2026

DANIELA ARALDI

**GERAÇÃO DE REGRAS *SIGMA* POR MEIO DE AGENTE AUTÔNOMO COM
RAG: ANÁLISE COMPARATIVA DE ESTRATÉGIAS**

**Sigma rules generation through LLM-based agent with RAG: a comparative
analysis of generation strategies**

Projeto de Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Bacharelado em Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Me. Adriano Serckumecka

Coorientador: Prof. Dr. Jefferson Tales Oliva

PATO BRANCO

2026



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

RESUMO

O resumo deve ressaltar de forma sucinta o conteúdo do trabalho, incluindo justificativa, objetivos, metodologia, resultados e conclusão. Deve ser redigido em um único parágrafo, justificado, contendo de 150 até 500 palavras. Evitar incluir citações, fórmulas, equações e símbolos no resumo. A referência no resumo é elemento opcional em trabalhos acadêmicos, sendo que na UTFPR adotamos por não incluí-la nos resumos contidos nos próprios trabalhos. As palavras-chave e as keywords são grafadas em inicial minúscula quando não forem nome próprio ou nome científico e separados por ponto e vírgula.

Palavras-chave: regras sigma; engenharia de deteco; llm; rag; automao em ciberseguran.

ABSTRACT

Seguir o mesmo padrão do resumo, com a tradução do texto do resumo e referência, se houver, para a língua estrangeira (língua inglesa).

Keywords: sigma rules; detection engineering; llm; rag; cybersecurity automation.

LISTA DE QUADROS

LISTA DE ABREVIATURAS E SIGLAS

Siglas

BGL	<i>BlueGene/L</i>
CNN	Rede Neural Convolucional, do inglês <i>Convolutional Neural Network</i>
CoT	<i>Chain-of-Thought</i>
CoT-R	<i>Chain-of-Thought + Reflection</i>
CVE	Vulnerabilidades e Exposições Comuns, do inglês <i>Common Vulnerabilities and Exposures</i>
DVWA	Damn Vulnerable Web Application, do inglês <i>Damn Vulnerable Web Application</i>
IOC	Indicador de Comprometimento, do inglês <i>Indicator of Compromise</i>
IR	<i>Intermediate Representation</i>
KQL	<i>Kusto Query Language</i>
LLM	Modelo de linguagem de grande escala, do inglês, <i>Large Language Model</i>
ML	Aprendizado de Máquina, do inglês <i>Machine Learning</i>
OSCTI	<i>Open-Source Cyber Threat Intelligence</i>
RAG	<i>Retrieval-Augmented Generation</i> , em português, Geração Aumentada por Recuperação
SIEM	Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês <i>Security Information and Event Management</i>
SPL	<i>Splunk Processing Language</i>
TF-IDF	Frequência de Termo–Frequência Inversa de Documento, do inglês <i>Term Frequency–Inverse Document Frequency</i>

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Motivação	7
1.2	Objetivos	8
1.2.1	Objetivo geral	8
1.2.2	Objetivos específicos	9
1.3	Estrutura do trabalho	9
2	REFERENCIAL TEÓRICO	10
2.1	Segurança da Informação e SOC	10
2.1.1	SIEM	10
2.1.2	SIEMs Inteligentes	11
2.2	Bases de Padrões de Ataques	12
2.3	Regras <i>Sigma</i>	13
2.3.1	Sigma e a mitigação de ameaças	14
2.3.2	Estrutura	15
2.4	Processamento de Linguagem Natural	17
2.4.1	Modelos de Linguagem de Grande Escala (LLMs)	17
2.4.2	Arquitetura Transformer	19
2.4.3	Engenharia de Prompt	19
2.5	Geração Aumentada por Recuperação (RAG)	19
2.5.1	<i>Embeddings</i> e Bancos de Dados Vetoriais	19
2.5.2	RAG aplicado à Cibersegurança	19
2.6	Agentes Autônomos	19
2.6.1	LangGraph como Orquestrador	19
3	TRABALHOS RELACIONADOS	20
3.1	<i>LLMCloudHunter</i>	20
3.2	<i>RulePilot</i>	21
3.3	GRIDAI	23
3.4	Aprendizado de máquina aplicado a SIEMs e análise de <i>logs</i>	24
3.5	Posicionamento do Trabalho Proposto	25
4	MATERIAIS E MÉTODOS	28

4.1	 Materiais	28
4.2	 Métodos	28
4.2.1	Primeira fase	28
4.2.2	Segunda fase	29
4.2.3	Terceira fase	30
5	 RESULTADOS	31
5.1	 Escopo do sistema	31
5.2	 Modelagem do sistema	32
5.3	 Apresentação do sistema	32
5.4	 Implementação do sistema	32
5.5	 Discussões (opcional)	33
6	 CONCLUSÃO	35
	 REFERÊNCIAS	36
	 GLOSSÁRIO	39

1 INTRODUÇÃO

1.1 Motivação

A segurança cibernética contemporânea representa a base da estabilidade civilizatória global e isto se agrava pela evolução constante das ameaças digitais e complexidade das infraestruturas tecnológicas modernas. Os ataques cibernéticos alimentam um ciclo vicioso de vulnerabilidades, comprometendo desde a integridade de dados sensíveis até a continuidade de serviços essenciais, como por exemplo nas áreas da saúde, energia e finanças. Este ciclo demanda abordagens inovadoras diante dos desafios crescentes em face às sofisticações dos ataques e da expansão das infraestruturas tecnológicas heterogêneas (Shah *et al.*, 2022).

Por infraestrutura heterogênea, entende-se um ambiente composto de diferentes tipos de tecnologias, dispositivos, sistemas operacionais, plataformas e arquiteturas que coexistem e interagem entre si. Tal heterogeneidade aumenta a complexidade de gestão e de segurança, pois cada componente possui vulnerabilidades específicas, demandando atualizações distintas e estratégias diferentes de proteção. Neste cenário, um único ponto fraco pode comprometer todo o ecossistema. Portanto, isso reforça a ideia de que a defesa cibernética precisa ser cada vez mais inovadora e adaptativa (Shah *et al.*, 2022; PALO ALTO NETWORKS, 2025; EXABEAM, 2025).

Os Sistemas de Gerenciamento de Informações e Eventos de Segurança (SIEMs) têm se consolidado como pilares na detecção e resposta a ameaças cibernéticas, especialmente diante da sofisticação dos ataques e do volume exponencial de dados gerados — um exemplo são os *logs*, registros automáticos de eventos que ocorrem em sistemas, aplicativos, redes ou dispositivos, gerados diariamente dentro de uma empresa. SIEMs, de forma similar aos *firewalls*, IDSs (Sistemas de Detecção e Intrusão), IPSs (Sistemas de Prevenção de Intrusão) e *syslogs*, são baseados em regras para filtrar e detectar ameaças e invasões. A má notícia é que essas regras tradicionais de detecção nem sempre acompanham a evolução das técnicas maliciosas, apresentando limitações diante da necessidade de respostas mais rápidas e inteligentes (Shah *et al.*, 2022; Podzins; Romanovs, 2019).

Dentre as iniciativas que buscam suprir essa lacuna, destacam-se as regras *Sigma*, um formato aberto e descritivo voltado à criação de regras de detecção independentes de plataformas, o que facilita sua adoção em diferentes soluções de SIEMs. No entanto, a criação e manutenção manual dessas regras pode se tornar um gargalo, principalmente em ambientes com elevado volume de eventos de segurança (SIGMAHQ, 2025a).

O padrão *Sigma* permite a criação de regras de detecção em YAML — uma linguagem de serialização de dados, usada para representar informações estruturadas de forma simples e legível, sendo compartilháveis e convertíveis para linguagens como *Splunk SPL* ou *Elasticsearch Query* (linguagens de consulta nativas respectivamente do *Splunk* e *Elasticsearch*), otimizando a colaboração entre as equipes de segurança. No entanto, a geração manual dessas regras

ainda é lenta e propensa a lacunas, especialmente diante de ameaças emergentes, como um possível *ransomware* (tipo de *malware* que sequestra dados ou sistemas e exige resgate para devolvê-los ao usuário) ou *zero-day exploit* — ataque em que se explora uma falha de segurança desconhecida do *software*, *firmware* ou *hardware* quando a vulnerabilidade ainda não foi descoberta pelo desenvolvedor e não há correção disponível (SIGMAHQ, 2025a; EXABEAM, 2025).

Com isso, é evidente que a aplicação de técnicas de inteligência artificial (IA) é uma ferramenta promissora na automatização da geração de regras *Sigma*, agilizando a detecção de comportamentos suspeitos ou maliciosos, considerando o tempo útil da elaboração de uma regra e a sua qualidade técnica.

Sendo assim, este trabalho propõe a implementação de um agente capaz de automatizar a criação de regras *Sigma*, a partir de vulnerabilidades e dados históricos de ataques — s, indicadores de comprometimento (IOCs) e *logs* de incidentes. A ideia principal é a ferramenta estar habilitada para criar regras pontuais, com base em nova notícia sobre vulnerabilidade reproduzida na *internet*, CVE, CWE, de forma supervisionada pelo analista (nada será totalmente automático), além de frequentemente buscar e sugerir atualizações para as regras utilizadas pelo usuário conforme novas informações sejam publicadas, de forma a melhorar sua acurácia.

O agente utilizará técnicas de Processamento de Linguagem Natural (NLP) para extrair padrões de ameaças em textos não estruturados, como relatórios de ataques e artigos técnicos (Shah *et al.*, 2022), e técnicas de aprendizado de máquina — mais precisamente LLMs (*Large Language Models*) para a geração autônoma de regras *Sigma*. O estudo de Lempinen, Pyyny e Juntunen (2023) apud Hasanov *et al.* (2024) mostra que na integração entre o *ChatGPT* e o SIEM *Wazuh*¹ houve eficácia na análise de *logs*, detecção de padrões de ameaças e automização de respostas de segurança. O *feedback* dos usuários indicou que a LLM é fácil de usar e eficaz no fornecimento de informações detalhadas sobre segurança, beneficiando principalmente os usuários com conhecimento limitado sobre o assunto. Esse tipo de capacidade sugere que LLMs podem ser adaptados para gerar regras *Sigma* autônomas a partir de dados de ameaças em tempo real.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver um agente que automatize a criação de regras *Sigma* para SIEM, treinado com a integração de dados históricos de ataques cibernéticos (IOCs, CVEs, *logs* de incidentes)

¹ O *Wazuh* é uma solução de segurança de código aberto que possui funcionalidades de HIDS (*Host-based Intrusion Detection System*, Sistema de Detecção de Intrusão baseado em Host) e SIEM (*Wazuh*, Inc., 2024).

e vulnerabilidades, visando otimizar a detecção de ameaças em SIEMs e reduzir a demora entre a identificação de uma ameaça e sua mitigação.

1.2.2 Objetivos específicos

- Criação de regras *Sigma* utilizando LLMs;
- Desenvolvimento de um agente autônomo para a geração de regras *Sigma*;
- Comparação dos resultados dos métodos utilizados;

1.3 Estrutura do trabalho

Este trabalho possui cinco capítulos, organizados de forma a conduzir o leitor do embasamento teórico à abordagem metodológica que será adotada. No primeiro capítulo, apresenta-se a introdução do tema, defendendo a motivação, os objetivos e a justificativa do estudo. O capítulo 2 disserta sobre conceitos necessários para se compreender o assunto do trabalho: sistemas SIEM, regras *Sigma* e LLMs — o que são, como se configuram na segurança da informação e como se relacionam entre si diante da mitigação de ameaças. Já o capítulo 3 discute trabalhos relacionados, oferecendo uma visão crítica sobre pesquisas e soluções correlatas. O capítulo 4 descreve os materiais e os métodos utilizados, detalhando as etapas do desenvolvimento da proposta em três fases. Por fim, são apresentadas as referências que fundamentam teoricamente o trabalho.

2 REFERENCIAL TEÓRICO

Neste capítulo será discorrido sobre os principais conceitos que justificam o escopo deste trabalho, como o que são e qual a importância dos SIEMs na área de segurança da informação, o que são as regras *Sigma*, o que são LLMs e como os assuntos se relacionam entre si.

2.1 Segurança da Informação e SOC

2.1.1 SIEM

Um Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês *Security Information and Event Management* (SIEM) funciona como o núcleo central de inteligência em ambientes de cibersegurança, agregando e correlacionando *logs* provindos de toda a infraestrutura de TI — como *firewalls*, sistemas de detecção de intrusão (IDS), todos os tipos de servidores, bancos de dados e entre outras inúmeras aplicações. Sua essência reside na capacidade de contextualizar eventos aparentemente desconexos, identificando padrões que ferramentas isoladas não detectariam. Conforme destacam Podzins e Romanovs (2019), essa contextualização é crucial em ações automatizadas contra ameaças complexas e citam o exemplo de um ataque de *zero-day*, onde a correlação entre uma tentativa de força bruta no *Active Directory* (serviço de diretório da *Microsoft* que gerencia identidades, recursos e permissões dentro da rede interna) e a existência de tráfego suspeito no *firewall* pode disparar respostas imediatas.

Nos Centros de Operações de Segurança (SOCs), o SIEM assume um papel operacional indispensável. Ele consolida alertas de diferentes fontes e formatos, prioriza incidentes com base na criticidade e orquestra processos de resposta (*workflows*). Podzins e Romanovs (2019) enfatizam ainda que, sem um SIEM, os SOCs perdem a capacidade de detectar fases avançadas de ataques, como por exemplo, de movimento lateral e escalção de privilégios, que se caracterizam pela movimentação do invasor por meio dos sistemas do alvo enquanto adquire privilégios de acesso —visando o nível de administrador; é citado o caso da rede de hotéis *Starwood*, que permaneceu quatro anos sem identificar uma violação sequer e sofreu um vazamento de dados de 383 milhões de hóspedes ao redor do mundo em 2019 (Shepardson, 2019). Diante do exposto, essa visão unificada transforma o SIEM no “sistema nervoso” do SOC, permitindo que analistas investiguem ameaças sistêmicas, não apenas alertas isolados.

A eficácia de um SOC está intrinsecamente ligada à maturidade do SIEM. Regras mal calibradas geram excesso de falsos positivos; SIEMs não otimizados podem produzir mais de mil alertas diários, sobrecarregando as equipes — segundo Podzins e Romanovs (2019), uma pessoa sozinha é capaz de analisar apenas cerca de 5 a 15 alertas por dia, a depender da cri-

tidade dos mesmos. Por outro lado, uma configuração refinada permite que um único analista investigue milhares de *logs*, elevando exponencialmente a eficiência operacional. A empresa Exabeam reforça que essa otimização é a base para um SOC ágil e proativo (EXABEAM, 2025).

A importância estratégica de um SIEM convencional na cibersegurança moderna se ancora na sua capacidade de reduzir drasticamente o tempo de detecção de ameaças, principalmente porque, conforme o passar dos anos, qualquer tipo de dado se tornou o recurso mais valioso que existe. Podzins e Romanovs (2019) citam um relatório de vazamentos de dados de 2018 da Verizon (Business, 2018), informando que cerca de 68% das violações são descobertas meses (ou mais) após o ataque inicial; o último relatório, de 2025, não atualiza esta porcentagem, mas informa que o tempo médio de remediação de vulnerabilidades nos dispositivos de borda (especialmente falhas de *zero-day*) é de 32 dias (Business, 2025). Este tipo de lacuna pode ser preenchida quando se tem um SIEM bem implementado, em virtude da correlação dos eventos em tempo real. Além disso, ele serve como alicerce para a evolução tecnológica: plataformas SIEM convencionais são a base necessária para a integração de IA e aprendizado de máquina, que posteriormente amplificam a detecção de ameaças sutis e permitem a automação de respostas (PALO ALTO NETWORKS, 2025).

»»Qual a diferença entre um SIEM e um antivírus tipo o Acronis?««««

Economicamente, embora o SIEM convencional envolva custos elevados (licenças baseadas em volume de *logs*, equipe especializada e SOC ininterrupto), Podzins e Romanovs (2019) justificam seu investimento frente ao cenário de riscos globais: projetou-se US\$ 5,2 trilhões em perdas por ciberataques entre 2019 — 2023. Para organizações com dados sensíveis ou requisitos de conformidade, como o GDPR — Regulamento Geral sobre a Proteção de Dados da União Europeia, ou a PCI-DSS — *Payment Card Industry Data Security Standard*, norma globalmente reconhecida aplicada à segurança de dados de cartões de pagamento, o SIEM é não apenas viável, mas indispensável; sua ausência representa um lapso operacional intransponível, mesmo com outras soluções de segurança.

Em suma, o SIEM convencional se mantém como pilar irrevogável da segurança corporativa; não por sua tecnologia isolada, mas também por habilitar ecossistemas integrados de detecção e resposta. Conclui-se que sua implementação estratégica, ainda que desafiadora, é um divisor de águas entre organizações expostas a violações prolongadas e aquelas capazes de antecipar, conter e remediar ameaças em escala evolutiva.

2.1.2 SIEMs Inteligentes

A integração de aprendizado de máquina (do inglês, *machine learning*, ML) e inteligência artificial (IA) em SIEMs transforma a cibersegurança ao superar as limitações das abordagens baseadas em regras estáticas. De acordo com Saddi *et al.* (2024), a IA generativa consegue criar dados sintéticos para simular ameaças inéditas, permitindo a detecção proativa de ataques *zero-day* sem depender exclusivamente de dados previamente rotulados. Essa capacidade é

amplificada pelos algoritmos de ML que analisam padrões contextuais em grandes volumes de *logs*, correlacionam eventos aparentemente desconexos (como tráfego anômalo com tentativas de força bruta) e identificam ameaças complexas. A predição de ameaças e a automatização dessas análises são fatores críticos para a atuação dos SOCs.

A integração de técnicas de IA e aprendizado de máquina em sistemas SIEM melhora significativamente a correlação de grandes volumes de eventos em tempo real, reduzindo falsos positivos ou falsos negativos, acelerando a triagem de alertas (PALO ALTO NETWORKS, 2025). Conforme demonstrado por Nurusheva *et al.* (2024), algoritmos de detecção de anomalias, análise preditiva e limiares adaptativos permitem identificar proativamente atividades sutis ou ocultas que escapariam sob regras estáticas ou análise manual. Ainda, ao empregar ambos modelos supervisionados e não supervisionados, o SIEM com ML aprende continuamente a partir dos dados de *logs* e do comportamento da rede, ajustando-se dinamicamente às mudanças no ambiente e antecipando novos vetores de ataque (Jordan; Mitchell, 2015).

É visto que, com o crescimento exponencial de dados e o aumento da complexidade das ameaças, é inviável depender exclusivamente de analistas humanos, tornando substancial o uso estratégico de ML/IA por parte dos SIEMs. Essa prática permite que o SIEM evolua de uma abordagem baseada em regras para uma abordagem comportamental e baseada em risco, onde perfis de usuários e entidades são criados automaticamente e usados para detectar desvios. Isso se alinha com o trabalho de Saddi *et al.* (2024), que propõem modelos de IA generativa para analisar fluxos de dados e gerar respostas simuladas a possíveis ataques, ajudando a antecipar e mitigar riscos de forma mais eficaz. Portanto, a incorporação de IA e ML em SIEMs representa um avanço significativo para a cibersegurança moderna, trazendo precisão, adaptabilidade e automação a um cenário cada vez mais dinâmico e hostil.

2.2 Bases de Padrões de Ataques

Para a criação de regras em SIEMs, assim como para qualquer tipo de regra, há de se adotar algum padrão. No mundo da cibersegurança existem padrões que catalogam e identificam vulnerabilidades, eventos e outras categorias do gênero. Duas categorizações são o CVE e o CWE. O CVE — *Common Vulnerabilities and Exposures* (Vulnerabilidades e Exposições Comuns), é um programa mantido pelo MITRE desde 1999, que define e cataloga vulnerabilidades e exposições de segurança publicamente conhecidas, no qual se atribui um identificador único (CVE-ID) e padrão para cada uma. Atualmente possuindo quase 300 mil registros, o CVE é uma referência confiável para integrar bancos de dados de segurança, alertas, produtos comerciais e soluções automatizadas (National Institute of Standards and Technology (NIST), 2025a; The MITRE Corporation, 2025a).

Já o CWE — *Common Weakness Enumeration* (Enumeração de Fraquezas Comuns), também mantido pelo MITRE, é um catálogo comunitário de tipos de fraquezas (como falhas de validação, *buffer overflow*, injeção), usadas para identificar a causa raiz subjacente às vulnerabi-

lidades. Quando um CVE descreve um “*buffer overflow*”, ele pode ser correlacionado a um CWE específico, permitindo que os analistas entendam quando se trata de um erro de lógica, design ou implementação (The MITRE Corporation, 2006–2025; The MITRE Corporation, 2025c).

O MITRE, que mantém as duas listagens citadas acima, é uma organização sem fins lucrativos, mantida por agências como o NIST (Instituto Nacional de Padrões e Tecnologia, do inglês *National Institute of Standards and Technology*) e CISA (Agência de Segurança Cibernética e Infraestrutura, do inglês *Cybersecurity and Infrastructure Security Agency*). Além da contribuição citada acima, o MITRE criou uma solução alternativa chamada MITRE ATT&CK — uma base de dados sobre táticas e técnicas adversárias, acessível globalmente, baseada em observações do mundo real. É usada como base para o desenvolvimento de modelos e metodologias de ameaças no setor privado, governo e na comunidade de produtos e serviços de cibersegurança (The MITRE Corporation, 2025d). A base é aberta e está disponível gratuitamente para uso de qualquer indivíduo ou organização, assim como o CVE e CWE.

Não menos importante é o NVD — *National Vulnerability Database* ou Base Nacional de Dados de Vulnerabilidades, repositório do governo dos EUA e mantido pelo NIST. É responsável por armazenar dados padronizados sobre vulnerabilidades de segurança seguindo o protocolo SCAP (*Security Content Automation Protocol*). O repositório inclui listas de falhas em *softwares*, métricas de impacto, nomes de produtos, configurações vulneráveis e *checklists*, permitindo automação da gestão de vulnerabilidades, avaliação de segurança e conformidade (National Institute of Standards and Technology (NIST), 2025b).

Esses elementos são pilares para as regras de SIEMs/*Sigma*, pois fornecem contexto de acionamento e padronização. O NVD enriquece CVEs com dados de severidades (como o CVSS (*Common Vulnerability Scoring System*), método usado para fornecer uma medida qualitativa de gravidade (National Institute of Standards and Technology (NIST), 2025c)) e produtos afetados, permitindo que SIEMs priorizem alertas com base no risco real. As classificações CWE ajudam a desenvolver regras *Sigma* para fraquezas comuns, detectando ataques mesmo quando variantes novas surgem. Já o mapeamento MITRE entre CWE e CVE vincula falhas de código a vulnerabilidades exploradas, refinando a correlação de ameaças em SIEMs. Por exemplo, uma regra *Sigma* pode usar o CWE-352¹ para monitorar solicitações não autorizadas, enquanto o CVE associado direciona *patches* críticos (NIST, 2024).

2.3 Regras *Sigma*

Sigma (*Generic Signature Format for SIEM Systems*, “Assinatura em formato genérico para sistemas SIEM” em tradução livre (SIGMAHQ, 2025b)) é o projeto de código aberto de um padrão genérico de descrição de *logs* de eventos, aplicável a qualquer tipo de *log*, com o intuito de compartilhar a lógica de detecção de ameaças ou anomalias entre diferentes SIEMs. Ao

¹ Falsificação de solicitação entre sites, do inglês *Cross-Site Request Forgery* (CSRF) (The MITRE Corporation, 2025b).

operar com dados de ameaças de múltiplas fontes, o *Sigma* capacita os caçadores de ameaças a correlacionar eventos que passariam despercebidos por abordagens convencionais. Dessa forma, se obtém conhecimentos estratégicos sobre dados confiáveis e antecipa-se tentativas de ataque em estágios iniciais desta cadeia de eventos (McGowan, 2025).

Os mantenedores da iniciativa consideram que excelentes análises podem ser encontradas na *internet*, incluindo IOCs e regras YARA² para detectar arquivos ou ferramentas maliciosos, por exemplo, mas que muitas vezes limitam-se a descrições específicas, funcionais para poucos *softwares*. Portanto, da necessidade de haver um método de descrição específico ou genérico de eventos de *logs*, com portabilidade para vários SIEMs, a fim de aprimorar os recursos de detecção para todos, surgiu o Sigma (SIGMAHQ, 2025b).

O projeto de código aberto foi criado em 2017 por Florian Roth e Thomas Patzke (SIGMAHQ, 2025a), inspirados em padrões já existentes nas áreas de análise de *malware* e monitoramento de rede. Como já dito, o padrão *Sigma* se diferencia por lidar com vários tipos de esquemas de *logs*, unindo padrões que já existem e se diferenciam entre si (McGowan, 2025).

O projeto carrega essa denominação pois se considera um ecossistema — envolve três componentes: o formato *Sigma*, a ferramenta *Sigma* e a coleção de regras *Sigma*. Este último diz respeito ao repositório no *Github*, que possui mais de 3 mil regras de detecção de diferentes tipos; por ser de código aberto, pode receber sugestões de qualquer pessoa disposta a contribuir (SIGMAHQ, 2025b). Já a ferramenta *Sigma*, refere-se a um conversor de regras disponível para *download* — o *Sigma Command Line Interface* (CLI). Com ele é possível converter as regras *Sigma* para as regras usadas no SIEM de interesse do usuário; *Elasticsearch*, *CrowdStrike*, *IBM QRadar AQL* e *Splunk* são algumas marcas registradas para as quais essa integração está disponível.

2.3.1 Sigma e a mitigação de ameaças

As regras *Sigma* formam um conjunto estruturado de princípios e métodos essenciais para o sucesso da investigação focada em determinar as ameaças que podem prejudicar um ou mais sistemas (*Threat Hunting*), oferecendo um padrão uniforme que orienta os profissionais de segurança na elaboração de regras criteriosas para detecção de ameaças ou anomalias. Segundo Dimitrov e Nikolov (2025), essas regras otimizam a identificação, avaliação e a resposta a incidentes, proporcionando decisões mais precisas e oportunas, além de uma base sólida para controlar vetores de ameaças, minimizar riscos e prevenir violações de dados. Dessa forma, as regras *Sigma* constituem o alicerce de uma estratégia de segurança proativa e resiliente, capaz de antecipar e mitigar possíveis ataques de forma sustentável.

A caça a ameaças é um processo defensivo proativo que visa descobrir e neutralizar invasores, mapeando vulnerabilidades e antecipando movimentos adversos por meio da análise

² Uma regra YARA é um arquivo de extensão “.yar” contido de expressões lógicas usadas para classificar e identificar amostras de *malware* (PICUS SECURITY, 2025).

contínua de comportamentos anômalos e incidentes em curso. Nesse cenário, as regras *Sigma*, desde sua padronização, em 2017, desempenham um papel fundamental ao oferecer uma linguagem comum para correlação de eventos em múltiplas plataformas SIEM, unificando dados de sistemas heterogêneos e acelerando a detecção de padrões suspeitos. Conforme demonstrado por Dimitrov e Nikolov (2025), essa unificação reduz significativamente a carga de trabalho manual dos analistas, permitindo identificar ameaças ainda na fase inicial de comprometimento, elevando a eficácia das operações de *Threat Hunting*, ou “caça a ameaças”.

No entanto, ainda que a aplicação das regras *Sigma* viabilize inúmeros cenários benquistos, vale ressaltar que, ainda assim, os SIEMs continuarão sendo heterogêneos, pois utilizam formatos e padrões de dados diferentes — situação inclusive de eventos de diferentes dispositivos. As regras *Sigma* surgem como uma forma de expressar a regra ou a ameaça e o que deve ser buscado, mas ainda precisa ser convertida para o SIEM de destino (através do CLI), que já possui suas próprias regras.

2.3.2 Estrutura

As regras *Sigma* são arquivos YAML contidos de informações que viabilizam a detecção de comportamento estranho ou malicioso quando da inspeção de arquivos de *logs* dentro do contexto de um SIEM (SIGMAHQ, 2025c).

A compreensão da estrutura de uma regra *Sigma* e como é usada para detecção faz-se importante em razão de que parte da metodologia envolverá selecionar as informações mais críticas das regras e avaliá-las.

As regras são divididas em três grupos principais de instruções (SIGMAHQ, 2025c):

- Detecção (*detection*): qual comportamento malicioso a regra está procurando;
- Fonte do log (*logsource*): quais tipos de logs a detecção deve procurar; e
- Metadados (*title*, *id*, *status*, etc.): detalhes da regra.

A Figura 1 é um exemplo de regra *Sigma* que está disponível na página oficial do projeto.

A seção de detecção é o coração da regra *Sigma*, pois especifica o alvo da regra no vasto volume de *logs*. Adiante neste trabalho este componente será mencionado como um dos principais para a análise da qualidade das regras geradas pelas IAs. Vale mencionar que, dentro da seção de detecção, existem instruções para a escrita, assim como qualquer linguagem de programação; no entanto, num primeiro momento, basta compreender o esqueleto principal. Sendo assim, a segunda peça mais importante da regra é a fonte dos *logs*, o campo *logsource*; nele se especifica qual o tipo de *log* deve ser canalizado pela regra — ao invés de varrer todos os *logs* do SIEM. Por fim, as demais informações, os metadados, servem para guiar o usuário sobre o assunto da regra (SIGMAHQ, 2025c).

Figura 1 – Exemplo de regra *Sigma* (SIGMAHQ, 2025c)

```

1  # ./rules/cloud/okta/okta_user_account_locked_out.yml
2  title: Okta User Account Locked Out
3  id: 14701da0-4b0f-4ee6-9c95-2ffb4e73bb9a
4  status: test
5  description: Detects when a user account is locked out.
6  references:
7    - https://developer.okta.com/docs/reference/api/system-log/
8    - https://developer.okta.com/docs/reference/api/event-types/
9  author: Austin Songer @austinsonger
10 date: 2021-09-12
11 modified: 2022-10-09
12 tags:
13   - attack.impact
14 logsource:
15   product: okta
16   service: okta
17 detection:
18   selection:
19     displaymessage: Max sign in attempts exceeded
20     condition: selection
21 falsepositives:
22   - Unknown
23 level: medium

```

Fonte: Captura de tela da página *Sigma Rules* em SigmaHQ (2025c).

Para fins de orientação quando da criação de uma nova regra, é disponibilizado um esqueleto da regra com instruções (Figura 2) no repositório *Sigma* (Roth, 2022).

Figura 2 – Exemplo de regra *Sigma* (Roth, 2022)

```

1  title: a short capitalised title with less than 50 characters
2  id: generate one here https://www.uuidgenerator.net/version4
3  status: experimental
4  description: A description of what your rule is meant to detect
5  references:
6    - A list of all references that can help a reader or analyst understand the meaning of a triggered rule
7  tags:
8    - attack.execution # example MITRE ATT&CK category
9    - attack.t1059      # example MITRE ATT&CK technique id
10   - car.2014-04-003    # example CAR id
11 author: Michael Haag, Florian Roth, Markus Neis # example, a list of authors
12 date: 2018/04/06 # Rule date
13 logsource:
14   category: process_creation # important for the field mapping in predefined or your additional config files
15   product: windows          # In this example we choose the category 'process_creation'
16   # the respective product
17 detection:
18   selection:
19     FieldName: 'StringValue'
20     FieldName: IntegerValue
21     FieldName|modifier: 'Value'
22   condition: selection
23 fields:
24   - fields in the log source that are important to investigate further
25 falsepositives:
26   - describe possible false positive conditions to help the analysts in their investigation
27 level: one of five levels (informational, low, medium, high, critical)

```

Fonte: Captura de tela do código genérico na página *Rule Creation Guide* no repositório Sigma(Roth, 2022).

2.4 Processamento de Linguagem Natural

Segundo Liddy (2001), o Processamento de Linguagem Natural (PLN) é definido como uma abordagem computacional teórica e tecnológica fundamentada para analisar textos em linguagem humana. Seu objetivo central é alcançar um processamento linguístico semelhante ao humano, através da representação de textos aos níveis linguísticos: fonológico (padrões de entonação e fonemas), morfológico (estrutura das palavras), léxico (significado individual das palavras), sintático (estrutura gramatical das frases), semântico (significado literal das frases), discursivo (coesão e estrutura dos textos) e pragmático (contexto); ou seja, o PLN fragmenta o texto e analisa sistematicamente as sete camadas citadas (Liddy, 2001; Chowdhary, 2020).

As técnicas de PLN evoluíram de métodos simbólicos, baseados em regras e lógica formal, para abordagens estatísticas e conexionistas (ou seja, inspirada no funcionamento do cérebro humano), impulsionadas pelos avanços computacionais e a disponibilidade de coleções extensas e estruturadas de textos (Liddy, 2001).

O PLN suporta aplicações críticas como a Recuperação de Informação (RI), Extração de Informação (EI), Tradução Automática (TA) e Sistemas de Pergunta-Resposta. Liddy (2001) cita um exemplo de RI em que sistemas PLN melhoraram a precisão em representar consultas e documentos em diversos níveis semânticos. A EI utiliza reconhecimento de entidades e análise parcial para estruturar dados de textos não estruturados, como em anúncio de emprego. Já os sistemas de diálogo, embora ainda dependentes de dimensões fonético-léxicas, buscam incorporar todas as camadas linguísticas para resultados mais naturais. A sumarização automatizada — geração de resumos concisos e coerentes a partir de textos extensos, aplica análise discursiva para condensar os textos enquanto mantém coerência (Chowdhary, 2020; Liddy, 2001).

Alguns dos obstáculos do PLN envolvem a integração de conhecimento de mundo e raciocínio de senso comum, essenciais para resolver ambiguidades contextuais complexas. Avanços nos modelos híbridos e ferramentas aceleram os experimentos, mas a compreensão profunda permanece distante, pois exige bases de conhecimento amplas e técnicas como inferência abductiva para superar lacunas de interpretação (Chowdhary, 2020).

2.4.1 Modelos de Linguagem de Grande Escala (LLMs)

Os Grandes Modelos de Linguagem, do inglês *Large Language Models* (LLMs), são sistemas de inteligência artificial treinados com bilhões ou trilhões de parâmetros, capazes de compreender e gerar textos em linguagem natural, ou seja, no formato de comunicação humano. A arquitetura desses modelos é baseada em “*Transformers*” — redes neurais baseadas em mecanismos de *self-attention* que permitem que o modelo foque em diferentes partes do texto simultaneamente, adquirindo relações contextuais de longo alcance entre as palavras (Weber *et al.*, 2024); estes aprendem em grande escala através do aprendizado auto supervisionado

(*self-supervised learning*), extraíndo padrões de grandes corpos de textos sem rótulos explícitos (Hasanov *et al.*, 2024). A evolução dessas redes marca um avanço na capacidade das máquinas “entenderem” contexto e intenção.

»»»»»»»»»»»»Motlagh, Jafari e Dehghantanha (2024) explica que o mecanismo central de um LLM se baseia na predição de *tokens* (menores unidades de texto que um modelo de linguagem divide para processar) sequenciais: dada uma sequência de entrada, o modelo estima a probabilidade do próximo elemento textual, refinando essa habilidade ao longo do pré-treino. Além disso, explica que técnicas como a “*in-context learning*” permitem que o mesmo modelo execute tarefas específicas apenas com os exemplos fornecidos pelo *prompt*, sem haver necessidade de ajuste fino. Esse paradigma torna possível a aplicação do mesmo modelo de LLM em diferentes tarefas sem precisar retreiná-lo (Motlagh; Jafari; Dehghantanha, 2024).

»»»»»»»»»»»»>A aplicabilidade dos LLMs é vasta: auxiliam na geração de código, criação de conteúdos, atendimento ao cliente, análise de sentimentos e suporte à decisão clínica, entre outras funções (Motlagh; Jafari; Dehghantanha, 2024). Contudo, desafios sempre existirão, e o alto custo computacional é um deles, além da tendência de reprodução de vieses presentes nos dados originais e vulnerabilidade a ataques de *jailbreak*³ e injeção de *prompt*⁴. Superar qualquer entrave exigirá trabalho contínuo, transparência dos dados e aperfeiçoamento incessante das técnicas de mitigação de riscos.

³ Processo de exploração de falhas de um dispositivo eletrônico bloqueado para instalar outro *software* que não o disponibilizado pelo fabricante para uso no dispositivo (Kaspersky, 2024).

⁴ Ocorre quando *prompts* do usuário alteram o comportamento ou a saída do LLM de maneiras não intencionais. Essas entradas podem afetar o modelo mesmo que sejam imperceptíveis para humanos. Portanto, as injeções de *prompt* não precisam ser visíveis/legíveis para humanos, desde que o conteúdo seja analisado pelo modelo. (OWASP Foundation, 2024)

2.4.2 Arquitetura Transformer

2.4.3 Engenharia de Prompt

2.5 Geração Aumentada por Recuperação (RAG)

2.5.1 *Embeddings* e Bancos de Dados Vetoriais

2.5.2 RAG aplicado à Cibersegurança

2.6 Agentes Autônomos

2.6.1 LangGraph como Orquestrador

3 TRABALHOS RELACIONADOS

3.1 *LLMCloudHunter*

O *LLMCloudHunter* é uma proposta de Schwartz *et al.* (2025), um *framework* que utiliza o modelo GPT-4o para automatizar a geração de regras *Sigma* a partir de relatórios de código aberto de inteligência de ameaças, voltados a ambientes em nuvem (*Open-Source Cyber Threat Intelligence* (OSCTI)). Os autores identificaram três limitações em trabalhos anteriores que motivaram a pesquisa: a ausência de saídas acionáveis, prontas para serem aplicadas em sistemas de segurança; o desperdício de informações visuais presentes em relatórios de ameaças, como capturas de tela e diagramas de fluxo de ataque; e o foco em ambientes *on-premises* (infraestrutura local), em detrimento dos ambientes em nuvem. Para superar essas lacunas, o *framework* utiliza uma abordagem multimodal, ou seja, com módulos que processam texto e imagens de forma conjunta.

A metodologia se organiza em três fases. Na fase de pré-processamento, a ferramenta *BeautifulSoup* é empregada no *web scraping* do código HTML do relatório, onde se descartam elementos irrelevantes – como barras laterais e propagandas; o conteúdo restante é convertido em um formato de texto unificado, no qual cabeçalhos, listas e tabelas têm sua estrutura hierárquica preservada. Em seguida, o componente *Image Analyzer* interpreta cada imagem do relatório por meio das capacidades visuais nativas do GPT-4o, transcreve seu conteúdo para texto e o reintegra à posição original no documento.

Na segunda fase o processamento é em nível de parágrafo, onde acontece a extração de entidades (pedaço de informação relevante) e a geração inicial dos candidatos à regra final. O componente *API Call Extractor* atua em duas etapas: primeiro extrai chamadas de API explicitamente citadas no texto e, em seguida, infere chamadas mencionadas de forma implícita. A inferência é necessária porque relatórios de ameaças frequentemente descrevem operações de alto nível que não correspondem diretamente a eventos registrados nos *logs*. Conforme ilustram os próprios autores, "descrições operacionais, como a realização de uma ação de sincronização (*sync*) em um *bucket* S3, devem ser mapeadas para as chamadas de API *ListBuckets* e *GetObject*" (Schwartz *et al.*, 2025, p. 1925). Para mitigar alucinações e omissões do modelo, esse componente incorpora um mecanismo de votação por maioria, executando a extração múltiplas vezes e mantendo apenas as ocorrências que ultrapassam um valor de referência pré-definido. Depois, o *TTP Extractor* mapeia cada chamada de API às respectivas táticas, técnicas e subtécnicas do MITRE ATT&CK, e o *Rule Generator* sintetiza essas informações na estrutura sintática das regras *Sigma*. Por fim, o *Rule Validator* higieniza os candidatos gerados, remove campos excessivamente específicos e ajusta a sintaxe ao padrão *Sigma*.

Na terceira fase, de processamento em nível de OSCTI, o escopo de refinamento é global – o *framework* olha todos os candidatos juntos para tomar decisões. O *Rule Optimizer* executa duas operações lógicas: unificação, que mescla campos de seleção que compartilham

os mesmos critérios de detecção; e separação, que divide campos com lógicas divergentes incorretamente agrupados. Em seguida, o *Set Refiner* elimina redundâncias decorrentes do processamento paralelo dos parágrafos, selecionando, para cada chamada de API repetida em múltiplos candidatos, a regra mais adequada e descartando ou ajustando as demais. Paralelamente, o *IoC Extractor* identifica endereços IP e *User Agents* citados no relatório, e em seguida o *IoC Enhancer* os integra ao conjunto final como campos de seleção adicionais, conectados à lógica original por meio de operadores condicionais, sem comprometer a estrutura de detecção previamente estabelecida.

Os resultados de Schwartz *et al.* (2025) demonstram que, na identificação das chamadas de API (sequência de comandos) atribuídas ao agente da ameaça nos relatórios analisados, o *framework* acertou 83% das extrações e recuperou 99% das ocorrências presentes nos relatórios. Para os indicadores de comprometimento (Indicador de Comprometimento, do inglês *Indicator of Compromise* (IOC)), os índices foram de 99% e 97%, respectivamente. A avaliação foi conduzida sobre uma base de vinte relatórios reais de ameaças em nuvem, previamente anotados de forma manual pela equipe de pesquisa. Além disso, 99,18% das regras *Sigma* geradas pelo *framework* foram compiladas com sucesso e convertidas em consultas para a plataforma *Splunk*, o que indica que a saída produzida atende ao padrão esperado. O estudo dos autores confirmou ainda a importância do processamento das imagens contidas nos relatórios, evidenciando que a leitura conjunta de texto e elementos visuais é uma estratégia adequada para lidar com a diversidade de formatos encontrados em relatórios de ameaças em nuvem (Schwartz *et al.*, 2025, seção 4.3).

3.2 RulePilot

O *RulePilot*, derivado do estudo de Wang *et al.* (2026), trata-se de um agente baseado em Modelo de linguagem de grande escala, do inglês, *Large Language Model* (LLM) para geração e conversão automatizada de regras de detecção em sistemas Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês *Security Information and Event Management* (SIEM)s proprietários, com foco principal em regras escritas na linguagem do *Splunk* (*Splunk Processing Language* (SPL)), com suporte adicional à linguagem do *Microsoft Sentinel* (a *Kusto Query Language* (KQL)). Nas motivações do trabalho, os autores justificam o foco em regras específicas de cada SIEM sob a argumentação de que formatos genéricos como o *Sigma*, apesar de sua portabilidade entre diferentes plataformas e fornecedores, dependem majoritariamente de correspondência de padrões por expressões regulares e não suportam consultas complexas como agregações estatísticas, junções multi-fonte ou cálculos em janelas temporais, tendo, portanto, limitações na eficácia de detecção de ataques sofisticados que envolvem sequências de eventos. Wang *et al.* (2026) citam como referência de trabalhos anteriores de geração automática de regras o próprio *LLMCloudHunter* (Schwartz *et al.*, 2024) – citado na seção anterior do presente trabalho. Os autores sustentam que *frameworks* como o citado contém tais limitações

por restringirem-se ao formato *Sigma*. Portanto, a motivação central do *RulePilot* é suprir a lacuna entre descrições de alto nível em linguagem natural e as regras complexas, específicas de cada fornecedor, requeridas por SIEMs em produção.

A arquitetura do *framework RulePilot* é organizada em três componentes principais. O primeiro é o módulo de raciocínio em cadeia (*Chain-of-Thought* (CoT)), no qual a descrição de entrada é decomposta em seis etapas estruturadas: interpretação dos objetivos de segurança, identificação das fontes de *logs*, definição de filtros iniciais, extração de campos relevantes, agregação de dados e otimização da regra. O segundo é a representação intermediária (*Intermediate Representation* (IR)), uma abstração estruturada que separa a lógica semântica da sintaxe específica do SIEM-alvo: `<KEYWORD> | PARAMS {k_i=v_i} | MODULES {m_j}`; *KEYWORD* designa a função da etapa (como *FILTER*, *EXTRACT*, *AGGREGATE* e *JOIN*); *PARAMS* define as configurações obrigatórias e *MODULES* descreve a intenção analítica. Essa separação permite que o LLM se concentre no raciocínio semântico antes de se comprometer com a sintaxe final. O terceiro componente é o mecanismo de reflexão e otimização iterativa, que avalia a regra gerada em três dimensões – consistência lógica, correção sintática e viabilidade de execução, e dispara dois tipos de refinamento: regeneração de etapas da cadeia de raciocínio e a integração com a API do *Splunk* real para validar com execução de *logs* verdadeiros. Para a funcionalidade de conversão entre linguagens SIEM, o *RulePilot* incorpora ainda um mecanismo de *Retrieval-Augmented Generation*, em português, Geração Aumentada por Recuperação (RAG) alimentado pela documentação oficial de migração da *Microsoft*, que mapeia comandos do SPL para seus equivalentes em KQL por meio de busca vetorial combinada à re-ranqueamento lexical. O *RulePilot* foi avaliado utilizando três LLMs distintos como motor – GPT-4o, DeepSeek-V3 e LLaMA-3, sobre 1.699 regras oficiais do repositório *Splunk Security Content* e 229.968 *logs* coletados de 61 testes atômicos do MITRE ATT&CK.

Os resultados de Wang *et al.* (2026) afirmam que o *RulePilot* supera os LLMs autônomos (sem o *framework*) em até 107,4% na similaridade textual com as regras de referência, medida pelas métricas BLEU, ROUGE-1, ROUGE-L e METEOR¹. Na avaliação de execução real sobre o *Splunk*, o *RulePilot* atingiu precisão de 100% em diversas táticas do MITRE ATT&CK, incluindo reconhecimento, acesso inicial e coleta, com desempenho superior ao MITRE-4o autônomo. Tais ganhos, contudo, acompanham certo custo computacional: o *RulePilot* consome cerca de dez vezes mais *tokens* de *prompt* e aproximadamente seis vezes mais tempo de geração que o LLM cru, em razão de seu raciocínio em múltiplas etapas e dos ciclos de refinamento iterativo. O estudo de ablação serve para entender o papel de cada parte do *RulePilot*. A ideia é simples: tira-se uma parte de cada vez para ver o quanto o resultado piora, assim se descobre para quê serve cada uma. O *RulePilot* tem duas partes principais: a primeira é a representação

¹ BLEU (Papineni *et al.*, 2002), ROUGE (Lin, 2004) e METEOR (Banerjee; Lavie, 2005) são métricas automáticas que medem a similaridade entre um texto gerado e uma referência pela sobreposição de n-gramas. BLEU prioriza precisão e ROUGE prioriza revocação; METEOR acrescenta sinônimos, radicais e ordem das palavras. Valores mais altos indicam maior alinhamento textual.

intermediária (IR), que ajuda a montar a regra com a sintaxe certa; quando ela é removida, o produto é uma regra com mais erros de escrita e de difícil leitura – a nota média cai de 0,55 para 0,44. A segunda parte é o raciocínio em cadeia com revisão (*Chain-of-Thought + Reflection* (CoT-R)) – ela ajuda a regra a ter sentido do começo ao fim. Quando se remove este bloco de raciocínio em cadeia e reflexão, o que mais piora é a consistência lógica e a cobertura de condições. A nota média cai para 0,41. Quando as duas partes são removidas juntas, o resultado é o pior de todos: a nota cai para 0,35. Ou seja, as duas se completam – cada uma cuida de um problema diferente. No fim, os autores testaram se o *RulePilot* consegue traduzir regras de uma plataforma para outra. Em regras simples, ele acertou tudo (nota 1,000). Em regras mais complicadas, que juntam dados de várias fontes, acertou quase tudo (nota 0,919). Isso mostra que dá para migrar regras de um sistema para outro sem grandes perdas. Por fim, os autores avaliaram a conversão entre linguagens SIEM, e o *RulePilot* atingiu nota 1,000 em F1, nas regras de agregação e de listagem. Já nas regras com junções de várias fontes, que são mais complexas, o F1 foi de 0,919. Esse resultado mostra que a migração entre plataformas SIEM diferentes é tecnicamente viável.

3.3 GRIDAI

O estudo de Li *et al.* (2025) (ainda apenas disponível como pré-print) propõe o GRIDAI, um *framework* de ponta a ponta para geração e reparo automatizado de regras de detecção de intrusão em sistemas baseados em assinaturas, como o Suricata e Snort. São citados alguns trabalhos anteriores que buscaram automatizar a criação dessas regras, como AutoCombo (Du; Hu; Hewlett, 2021) e Syrius (Alcantara *et al.*, 2021); entretanto, Li *et al.* (2025) argumentam que limitação central deles é que todos geram uma regra nova para cada amostra de ataque recebida e precisam de tráfego de rede normal como referência, sem nunca verificar se uma regra já existente poderia simplesmente ser atualizada. Com o tempo, isso gera um acúmulo de regras redundantes e um volume crescente de alertas duplicados, dificultando a manutenção do sistema. O GRIDAI propõe uma lógica diferente: antes de agir, ele avalia a natureza de cada amostra. Se o ataque for inédito, cria uma nova regra; se for uma variação de algo já conhecido, atualiza a regra existente para que ela passe a cobrir também esse novo caso. A arquitetura é composta por quatro módulos especializados coordenados por uma lógica central. O primeiro módulo (*Relation-Assess Agent*) funciona em dois passos: primeiro testa a amostra diretamente no motor de detecção e, somente se nenhum alerta for gerado, consulta o modelo de linguagem para decidir o que fazer. O segundo (*New-Rule-Generation Agent*) cria regras novas analisando o conteúdo do ataque em etapas, gerando e testando múltiplos candidatos. Já o terceiro (*Existing-Rule-Repair Agent*), ajusta regras existentes e garante que a versão modificada continue funcionando tanto para o ataque original quanto para a nova variante. E o quarto (*Memory-Update Agent*) registra todas as alterações. Para lidar com erros gerados pelos

modelos de linguagem, cada regra produzida é validada em tempo real no motor de detecção; se as falhas ultrapassarem um limite pré-definido, o sistema desfaz a operação e tenta novamente.

Os experimentos foram realizados em dois conjuntos de dados – um sintético (com sete tipos de ataques) e um real (com quarenta e três tipos), além de usar o GPT-4.1 como modelo principal. O sistema também foi testado com outros quatro modelos de linguagem (GPT-4o, GPT-3.5-turbo, GLM-4-Plus e GLM-4-Flash), demonstrando que o GRIDAI tem um bom funcionamento na utilização de modelos distintos. Quando na tarefa de identificar corretamente o que fazer com cada amostra, o sistema acertou 100% das decisões no conjunto sintético e 96,9% dos casos de reparo no conjunto real; a acurácia para geração de novas regras foi de 60,5%, em virtude de algumas regras do conjunto de referência compartilharem características muito similares entre si. Frente aos três trabalhos tomados como base de comparação por Li *et al.* (2025), o GRIDAI detectou 100% dos ataques no conjunto sintético e 95,4% no conjunto real, sem gerar nenhum falso alarme. O experimento de ablação, em que os autores removeram individualmente cada um dos principais mecanismos do sistema para medir o impacto de cada componente no desempenho final, mostrou que sem o mecanismo de reparo, a taxa de alertas duplicados sobe de 12,7% para 66,8%, confirmando que reaproveitar e atualizar regras existentes é muito mais eficiente do que criar novas regras a cada variante detectada.

3.4 Aprendizado de máquina aplicado a SIEMs e análise de logs

A aplicação de aprendizado de máquina para análise automatizada de *logs* e a identificação de ameaças foi explorada de forma convergente por Shah *et al.* (2022) e Nurusheva *et al.* (2024). Ambos trabalhos partem da constatação de que o volume e a heterogeneidade dos *logs* gerados por sistemas modernos inviabilizam a inspeção manual, e as abordagens tradicionais baseadas somente em regras estáticas falham na identificação de ataques novos ou ofuscados ². Para superar essa limitação, os dois estudos recorrem a técnicas de Aprendizado de Máquina, do inglês *Machine Learning* (ML), embora com metodologias distintas – empírico-quantitativa com foco em agrupamento no primeiro trabalho e com foco arquitetural e demonstrativo no segundo.

Shah *et al.* (2022) propõe um modelo que enfrenta o desafio da ausência de dados rotulados, problema recorrente em cenários reais de monitoramento. A metodologia inicia com pré-processamento dos registros, mantendo apenas o conteúdo textual das mensagens. Em seguida, os autores aplicam a vetorização Frequência de Termo–Frequência Inversa de Documento, do inglês *Term Frequency–Inverse Document Frequency* (TF-IDF) e executam o algoritmo de *clustering* não-supervisionado *K-means*. A partir dos agrupamentos obtidos, os pesquisadores rotularam os registros e geraram um conjunto de dados para treinar um classificador supervisionado *Naive Bayes*. Avaliado sobre o *dataset BlueGene/L* (BGL), o modelo obtido al-

² Ofuscação é a técnica de mascarar a aparência de um código, comando ou tráfego de rede para torná-lo incompreensível aos sistemas de detecção e analistas, sem alterar a função executável original.

cançou acurácia média de 98% na predição de eventos anômalos, demonstrando viabilidade para ambientes que exigem triagem rápida.

Já Nurusheva *et al.* (2024), propõem a integração direta de algoritmos com ML supervisionado a um sistema SIEM. A validação foi conduzida em uma arquitetura construída sobre a pilha ELK, com cenários de ciberataques simulados na aplicação propositalmente vulnerável chamada Damn Vulnerable Web Application, do inglês *Damn Vulnerable Web Application* (DVWA). Os autores treinaram modelos clássicos – *Support Vector Machine*, *Random Forest*, *Decision Tree* e *AdaBoost*, além de uma Rede Neural Convolucional, do inglês *Convolutional Neural Network* (CNN) adaptada à classificação de eventos de segurança, utilizando amostras de ataques baseadas no *framework* MITRE. Como resultado, relataram ganhos operacionais: a correlação de eventos reduziu falsos positivos e a automação da resposta diminuiu a carga sobre os analistas de segurança. Diferentemente do estudo anterior, o trabalho de Nurusheva *et al.* (2024) possui caráter predominantemente descritivo, com foco na arquitetura operacional, em detrimento de uma avaliação baseada em métricas comparativas rigorosas.

Ao analisar conjuntamente as duas pesquisas, vê-se um dilema prático na modernização e evolução da segurança cibernética: o equilíbrio entre o refinamento algorítmico isolado e a aplicabilidade sistêmica. Enquanto a abordagem de Shah *et al.* (2022) comprova que é possível contornar a escassez de dados rotulados por meio de *pipelines* híbridos de aprendizado de máquina, alcançando alta acurácia em ambientes controlados, a proposta de Nurusheva *et al.* (2024) evidencia que o verdadeiro desafio está na orquestração contínua desses modelos dentro de ecossistemas corporativos reais. Portanto, a consolidação do uso de aprendizado de máquina na triagem de eventos de segurança permanece condicionada à superação dessa fronteira: trabalhos como o de Shah *et al.* (2022) demonstram a viabilidade técnica do componente preditivo, enquanto Nurusheva *et al.* (2024) ilustram a integração operacional desses componentes em SIEMs – mas raramente os dois aspectos são tratados de forma articulada em um único estudo.

3.5 Posicionamento do Trabalho Proposto

O conjunto de trabalhos analisados nas seções anteriores expõe duas vertentes complementares na pesquisa em automação da detecção de ameaças. A primeira vertente se concentra em técnicas de aprendizado de máquina aplicadas à triagem e classificação de *logs* de segurança, sem produzir artefatos sintáticos diretamente acionáveis em sistemas SIEM. Nesta vertente encontram-se Shah *et al.* (2022) e Nurusheva *et al.* (2024), que combinam métodos supervisionados e não supervisionados para reduzir a carga de alertas em ambientes corporativos – estabelecendo, ainda que indiretamente, um terreno metodológico sobre o qual as abordagens mais recentes puderam se apoiar. A segunda vertente se concentra na geração automatizada das próprias regras de detecção, valendo-se das capacidades generativas dos LLMs, e representa uma evolução natural da primeira: uma vez consolidada a aplicabilidade de

aprendizado de máquina e inteligência artificial na análise de eventos de segurança, abre-se caminho para que essas mesmas técnicas atuem não apenas na classificação dos eventos, mas também na produção dos próprios mecanismos de detecção. Nesta segunda vertente está o GRIDAI ((Li *et al.*, 2025)), voltado para regras de sistemas de detecção de intrusão de rede; o *RulePilot* ((Wang *et al.*, 2026)), voltado para regras nativas das linguagens SPL e KQL; e o *LLMCloudHunter* ((Schwartz *et al.*, 2025)), voltado para regras *Sigma* em ambientes de nuvem. O presente trabalho melhor se insere na segunda vertente, compartilhando com Schwartz *et al.* (2025) o objetivo de geração de regras *Sigma*.

A análise comparativa dos estudos revela três observações fundamentais para o posicionamento da presente pesquisa. Num primeiro momento, dentre os cinco trabalhos analisados, apenas o *LLMCloudHunter* de Schwartz *et al.* (2025) compartilha exatamente o mesmo formato-alvo de saída (regras *Sigma*). No entanto, mesmo neste caso, há diferenças relevantes de escopo: o *LLMCloudHunter* opera sobre relatórios OSCTI como entrada e produz regras específicas à detecção em ambientes de nuvem AWS, enquanto o presente trabalho opera a partir de registros Vulnerabilidades e Exposições Comuns, do inglês *Common Vulnerabilities and Exposures* (CVE) estruturados e notícias de vulnerabilidade, visando a produção de regras *Sigma* não restritas a um provedor de nuvem específico. Em segundo, os trabalhos voltados a outras linguagens de regra – Li *et al.* (2025) e Wang *et al.* (2026), não competem diretamente no domínio *Sigma*, mas oferecem precedentes arquiteturais relevantes para a estruturação do agente proposto neste trabalho. O uso de validação iterativa em motor de detecção real ((Li *et al.*, 2025)) e o uso de raciocínio em cadeia com reflexão ((Wang *et al.*, 2026)) são padrões metodológicos que se aproximam da arquitetura LangGraph empregada nesta pesquisa, ainda que a aplicação tenha contextos sintáticos distintos. Em terceiro, os trabalhos da primeira vertente – Shah *et al.* (2022) e Nurusheva *et al.* (2024), embora não produzam regras como saída, são fundamentais para contextualizar a presente pesquisa: comprovam o quão útil é o uso de técnicas de aprendizado de máquina e inteligência artificial na detecção de ameaças em sistemas SIEM, fundamentando a premissa de que tais técnicas, hoje já consolidadas, estendem-se para tarefas sofisticadas como a geração automatizada de regras de detecção.

Há ainda uma divergência teórica interessante entre os trabalhos analisados, que merece saliência. Wang *et al.* (2026) argumentam que o formato *Sigma* é insuficiente para a detecção de ataques sofisticados que envolvem sequências de eventos, por restringir-se sobretudo à correspondência de padrões por expressões regulares, criticando trabalhos que adotam *Sigma* como saída – citando diretamente o *LLMCloudHunter*. O presente trabalho optou por manter o foco no formato *Sigma* em decorrência de sua portabilidade entre múltiplos desenvolvedores de SIEMs, da existência de um repositório público bem consolidado de regras (o SigmaHQ) que serve como base para o mecanismo de RAG, e da posição do formato como padrão aberto mantido pela comunidade de *detection engineering*. Essa escolha posiciona o trabalho aqui proposto em alinhamento com Schwartz *et al.* (2025), mas com clara consciência das limitações expressivas do formato apontadas por Wang *et al.* (2026). Vale ressaltar igualmente que a pró-

pria arquitetura proposta para o agente desenvolvido neste trabalho – fundamentada no uso de RAG sobre o repositório SigmaHQ, pode apresentar limitações na tarefa específica de geração de regras *Sigma*, conforme indícios observados ao longo do desenvolvimento. Portanto, investigação empírica conduzida neste trabalho de conclusão de curso pretende vislumbrar o quanto essa estratégia se sustenta frente às alternativas avaliadas, contribuindo para o esclarecimento dessa questão metodológica.

Ante o exposto, dessa análise emerge o recorte ocupado pelo presente trabalho. Enquanto o *LLMCloudHunter* demonstra a viabilidade técnica da geração de regras *Sigma* via LLM em um domínio específico, e enquanto GRIDAI e *RulePilot* exploram arquiteturas agênticas em outras linguagens de regra, permanece em aberto a questão metodológica de quanto a complexidade arquitetural – desde a invocação direta de uma LLM em formato *zero-shot* até a orquestração de um agente autônomo com RAG e validação iterativa, efetivamente contribui para a qualidade das regras *Sigma* geradas a partir de fontes públicas de inteligência sobre ameaças. O estudo aqui exposto investiga essa questão por meio da comparação controlada entre três estratégias incrementais — *zero-shot*, engenharia de *prompt* e agente autônomo com RAG – submetidas a teste estatístico de hipóteses. Diferentemente dos cinco trabalhos analisados, que tipicamente comparam uma única abordagem proposta contra LLMs autônomos ou métodos baseados em mineração de dados, o presente trabalho se propõe a quantificar empiricamente o ganho marginal proporcionado por cada camada de complexidade arquitetural na tarefa de geração de regras *Sigma*, fornecendo subsídios para decisões de *design* em trabalhos futuros na linha de pesquisa.

4 MATERIAIS E MÉTODOS

4.1 Materiais

Os materiais utilizados para o desenvolvimento deste trabalho serão essencialmente as regras *Sigma* (arquivos em YAML) disponíveis em seu repositório (SIGMAHQ, 2025b) — composto atualmente por mais de 3.400 regras; as LLMs mais conhecidas e de ampla utilização, como *ChatGPT*, *Deepseek*, *Microsoft Copilot*, *Gemini* e *Grok*; linguagem de programação *Python*; ambiente de desenvolvimento *Visual Studio Code*; para o desenvolvimento do agente, *framework* de código aberto *LangChain* — facilitador da construção de aplicações que usam modelos de linguagem grandes (LLMs) (LANGCHAIN, INC., 2025).

4.2 Métodos

A primeira fase envolveu a geração manual de regras utilizando as LLMs já mencionadas, visando um cenário simplificado e que remete a um usuário inexperiente. A segunda fase incluirá um *prompt* customizado que fornecerá dados e instruções relevantes para extrair melhor desempenho e acurácia das LLMs na geração das regras. Já na terceira fase será desenvolvido um agente especializado, treinado para a geração de regras de forma autônoma, com base nas referências/dados da vulnerabilidade da qual se pretende proteger.

Inicialmente serão obtidas todas as regras *Sigma* por meio de seu repositório no *GitHub* (SIGMAHQ, 2025b), sendo organizadas da seguinte forma: 3000 regras para a base de treinamento e 400 regras para testes. A avaliação dos resultados será realizada através de análise manual por amostragem e análise automatizada (Distância do Cosseno, *Term Frequency-Inverse Document Frequency* (TF-IDF), Jaccard - à definir) para verificar a acurácia perante as regras originais ou eventual melhoria em sua estrutura, por se tratar de uma busca mais recente, utilizando informações adicionais às referências usadas durante a criação da regra original. Por fim, uma análise de qual fase alcançou o melhor desempenho e qualidade será apresentada.

As etapas de desenvolvimento e avaliação do presente trabalho são detalhadas a seguir.

4.2.1 Primeira fase

Inicialmente serão selecionadas manualmente 10% das regras de teste (40); elas serão utilizadas para a geração de regras através de um *prompt* simples, sem customizações, em cada uma das LLMs, da mesma forma que um usuário inexperiente faria, conforme é exibido no “**Prompt Básico**”. Para solicitar a geração das regras, as referências de cada uma delas serão extraídas e armazenadas, já que contém o(s) *link*(s) com as informações fundamentais que

deram origem àquela regra. Em seguida serão incluídas no *prompt* e enviadas como requisição para a IA de forma manual.

Prompt Básico

“Generate a SIGMA rule (<https://sigmahq.io/docs/basics/rules.html>) for this event:
<https://localtonet.com/documents/supported-tunnels>
<https://cloud.google.com/blog/topics/threat-intelligence/unc3944-targets-saas-applications>”

Para fins explicativos, os *links* posteriores à palavra “*event*” são as referências inseridas na regra *Sigma* pelo seu autor (visíveis no campo “*reference*” das regras); e o *link* mencionado após a palavra “*rule*” é a página do repositório que instrui a escrita de uma regra *Sigma*.

O próximo passo será gerar as regras de forma automatizada, utilizando-se de um *script* para otimizar o processo. Tanto a seleção manual quanto a automatizada constituirá os resultados deste trabalho.

Após reunir réplicas¹ de regras *Sigma* já existentes, será realizada comparação manual por amostragem entre a regra original e suas réplicas, considerando os campos mais relevantes (que tornam a regra ativa ou operável no SIEM); são eles: os componentes *detection* e *logsource*.

Ao final dessa análise, o estudo demonstrará a qualidade das regras *Sigma* escritas através de um *prompt* básico e será possível indicar qual LLM se destacou percentualmente perante as demais.

4.2.2 Segunda fase

A segunda fase utilizará engenharia de *prompt* para fornecer às LLMs melhor contexto e informações precisas que aprimorem as regras desenvolvidas por elas, novamente resultando num estudo comparativo ranqueando quais devolvem melhores resultados.

O processo envolve o uso estratégico de palavras-chave, clareza na formulação dos textos, além de adaptação do *prompt* (Santos; Martins; Evangelista, 2024), pois segundo a própria OpenAI (2024), “diferentes tipos de modelos de linguagem podem precisar ser solicitados de forma diferente para produzir os melhores resultados”.

O processo de amostragem e teste da primeira fase será replicado, porém com o diferencial do refinamento dos *prompts*.

Sendo assim, as mesmas 40 regras de testes selecionadas serão utilizadas para a geração de regras através de um *prompt* customizado, em cada uma das LLMs – aqui o usuário é mais experiente que o anterior. Em seguida, utilizar-se-á de um *script* que automatize a tarefa.

¹ As réplicas são as regras geradas pelas IAs.

É importante salientar que em todas as fases será inclusa uma averiguação final de um especialista e/ou da “comunidade *Sigma*” (pessoas que contribuem regularmente com o repositório de regras). Ademais, os respectivos resultados passarão por análise qualitativa novamente e o método de comparação usado na primeira fase será repetido.

4.2.3 Terceira fase

Na terceira fase do projeto, o agente de IA será projetado e testado segundo sua precisão e qualidade das regras geradas. O desenvolvimento do agente começará por escrita de código e, quando preparado, será treinado com 3000 regras *Sigma*; para o teste serão 400. Ajustes serão feitos conforme necessidade. Das 400 réplicas por LLM (serão 2 mil no total, se forem usadas 5 LLMs), 10% serão retiradas para uma análise inicial manual, posteriormente automatizada.

Para as avaliações, primeiro será necessário definir parâmetros que validam uma regra. De antemão já pode-se considerar que as regras validadas serão, no mínimo, iguais às originais – o que não significa exatamente que são boas, pois algumas podem já estar datadas, por exemplo –, fazendo-se necessário verificar as regras descartadas para entender o motivo da não validação, pois algumas regras diferentes podem ser melhores que as originais. A avaliação das regras descartadas será feita por especialista ou pela comunidade *Sigma*.

Com os critérios definidos, os desempenhos dos LLMs serão medidos utilizando métricas qualitativas e quantitativas, calculadas para cada modelo e fase do experimento. Serão calculados, por exemplo, taxa de similaridade média textual, taxa de acerto e desvio padrão, analisando a proporção de réplicas consideradas “semelhantes” à original.

O produto final do trabalho será uma composição analítica qualitativa e quantitativa entre todos os produtos obtidos, das três fases.

5 RESULTADOS

Este capítulo apresenta o que foi obtido como resultado do trabalho, que, em princípio, é o sistema desenvolvido. Se não for um sistema, como, por exemplo, uma solução na área de redes, neste capítulo é reportada a solução proposta. Neste caso, a divisão do capítulo em seções é realizada, se necessária, de acordo com o trabalho.

O capítulo pode conter seções de acordo com o tipo de sistema e a necessidade de documentação mais extensa de determinados aspectos. Caso o trabalho se refira à comparação entre tecnologias ou dados obtidos como resultados do uso do sistema, além da descrição do sistema, há os dados obtidos com os testes e a discussão desses dados. Nesse caso haverá uma seção para os dados obtidos desses testes e as discussões.

5.1 Escopo do sistema

Apresenta o escopo do sistema (contendo entre dois ou cinco parágrafos) de forma bastante sucinta, considerando aspectos técnicos e conceituais. O escopo define o que é o sistema, consistindo das funcionalidades e características que o sistema deve conter. É importante apresentar também o escopo negativo, ou seja, as funcionalidades e características que o sistema não irá conter. Exemplo:

O sistema XYZ deve gerenciar todos os processos de uma livraria virtual, desde a aquisição até a venda dos livros para o consumidor final. O acesso dos compradores e gerentes deve ser feito por meio de um site WEB, incluindo a possibilidade de acesso por outras tecnologias (ex. celular, tablet). Os clientes poderão fazer as compras pagando com cartão de crédito ou depósito bancário. Existem promoções eventuais pelas quais os livros podem ser comprados com desconto.

De início, a livraria vai trabalhar apenas com livros novos a serem adquiridos de editoras que tenham sistema automatizado de aquisição. Desta forma, o sistema a ser desenvolvido deve conectar-se aos sistemas das editoras para efetuar as compras.

O sistema deve calcular o custo de entrega baseado no peso dos livros e na distância do ponto de entrega. Eventualmente podem haver promoções do tipo “entrega gratuita” para determinadas localidades.

O sistema deve permitir a um gerente emitir relatórios de livros mais vendidos, e compradores mais assíduos, bem como sugerir compras para compradores baseadas em seus interesses anteriores.

5.2 Modelagem do sistema

A modelagem do sistema inclui os diagramas e as descrições textuais para representar o problema e a solução.

Sendo assim, primeiramente esse item deve apresentar diagramas utilizados para a modelagem de negócios (ex. diagramas de atividade e estado), se esses tenham sido necessários. Em seguida esse item deve conter a descrição dos requisitos obtidos do usuário, contendo sua respectiva classificação (funcionais e não funcionais). Sugere-se o uso de um modelo formal sugerido por autores (ex. Wazlawick, Bezerra) para a apresentação dessa classificação.

Se utilizada orientação a objetos e a UML, nesta seção ainda são apresentados, por exemplo, os diagramas de casos de uso, com suas descrições suplementares, os diagramas de classe de análise (ou modelo conceitual), de sequência e/ou comunicação, diagrama de classes de projeto.

Nesta seção também estão os diagramas da modelagem de banco de dados, como entidade-relacionamento. Nesse item pode ser apresentada a descrição de cada uma das classes do modelo de classes apresentado acima, assim como a descrição das tabelas do banco de dados. Também podem estar documentados modelos e padronizações utilizados para a interface, diagramas de navegação, a representação da arquitetura do sistema e dos padrões de projeto utilizados.

5.3 Apresentação do sistema

Apresenta as funcionalidades e o uso de recursos tecnológicos do sistema por meio de suas telas, enfatizando a interação com o sistema. A apresentação do sistema é feita sob a forma de texto, com telas e definição de padrões que forem relevantes ao contexto do trabalho. As telas são tratadas como figuras, cópias (print screen) de relatórios ou consultas também são figuras.

A Figura 3 exibe a tela de acesso ao Cadastro de Pacientes.

Figura 3 – Tela de acesso ao Cadastro de Pacientes.

Fonte: Autoria própria (2026).

5.4 Implementação do sistema

Nesta seção é documentada a implementação do sistema com partes relevantes ou exemplos de código, rotinas, funções. Inclui, ainda, a descrição técnica do uso de recursos (componentes, bibliotecas, etc.) da linguagem. Ressalta-se que cada orientador avaliará juntamente com seu orientado o que poderá ser descrito nesta seção. Isso sem que sejam revelados

detalhes do sistema que possam comprometer seu uso comercial ou científico ou que a descrição fique muito sucinta ou superficial.

Em materiais e método estão quais os recursos utilizados, neste capítulo é reportado como esses recursos foram utilizados para resolver o problema.

Sugere-se colocar listagens curtas de código, enfatizando aspectos específicos das tecnologias utilizadas ou da implementação. Sugere-se, ainda, que o código não seja apresentado sob a forma de print screen, e sim copiado e colado no texto, mantendo, se possível, a formatação. Todas as listagens de código devem ser devidamente explicadas. A explicação deve ser técnica, fundamentada em aspectos conceituais e boas práticas de programação.

Enfatizar os diferenciais do sistema: procedimentos armazenados, consultas SQL, uso de componentes, uso de padrões de projeto, a forma de uso dos recursos da linguagem. Esses diferenciais são no sentido de explicitar as vantagens, desvantagens, dificuldades e facilidades que esses recursos impetraram no desenvolvimento do sistema em termos técnicos. Esses diferenciais servirão para avaliar pela utilização ou não desses recursos, pelo menos para sistemas iguais ou semelhantes ao reportado no trabalho.

Reportar a forma como o sistema foi verificado e validado. No sentido de verificar se os requisitos definidos para o mesmo foram atendidos. Os testes podem ser realizados pelo professor orientador, pelos professores que compõem a banca, por pessoas que serviram de base para as informações para o sistema e etc. Os testes podem ser realizados com base em um plano de testes elaborado juntamente com a análise e projeto do sistema. Para validar a implementação podem ser desenvolvidas rotinas de teste unitário.

Se houver implantação do sistema, mesmo que seja para teste, reportar a forma como isso foi feito, a geração de instaladores, os problemas com ambiente e sistema operacional, incluindo banco de dados e outros. Deixar explícito o procedimento para instalar e usar o sistema.

Quando for necessário, citar no texto do trabalho nomes de campos, tabelas ou rotinas específicas utilizadas na implementação de um software, utilizar a fonte courier new para destacar esses nomes.

Um exemplo de listagem de código fonte pode ser observado na Listagem 1, que representa a classe Aluno.

5.5 Discussões (opcional)

O trabalho contém esta seção quando considerado que há resultados (em termos de dados) e discussões relevantes ou suficientes para justificar uma seção. Se existentes e não justificarem uma seção, eles podem estar na seção que relata a implementação do sistema.

Nesta seção estão os resultados obtidos da realização de testes quantitativos e qualitativos, independentemente da quantidade, tipo e volume de testes realizados. Os resultados dos testes são discutidos tendo como base o referencial teórico e os objetivos pretendidos com o trabalho. Esses testes podem resultar de implantação e testes de uso do sistema.

Listagem 1 – Classe Aluno

```
1 @Entity
2 public class Foo {
3
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     private String nome;
9
10    private Integer ra;
11
12    // constructor, getters and setters
13 }
```

Fonte: Autoria própria (2026).

6 CONCLUSÃO

Inicia com um resumo do trabalho, retomando o(s) objetivo(s), o referencial teórico e o uso das ferramentas e das tecnologias utilizadas no trabalho.

A conclusão contém a opinião do autor em relação às vantagens, desvantagens, facilidades e limitações das tecnologias e/ou do método utilizados, as dificuldades encontradas e como foram superadas.

Também devem ser apresentadas as vantagens, desvantagens e limitações do trabalho desenvolvido, sempre tendo em vista a sua contribuição para a comunidade acadêmica e profissional e para a sociedade como um todo.

É a opinião técnica do autor do trabalho em relação ao assunto sob a forma de uma espécie de avaliação em relação ao trabalho desenvolvido e as tecnologias utilizadas.

Finaliza verificando se o objetivo foi alcançado e com a opinião do autor sobre o assunto, de acordo com o referencial teórico e com os resultados obtidos.

As perspectivas futuras são opcionais, devem ser apresentadas somente caso o acadêmico pretenda dar continuidade ao trabalho, ou mesmo se ele julgar relevante que outras pessoas dêem continuidade ao seu trabalho.

REFERÊNCIAS

- ALCANTARA, L. *et al.* Sirius: Synthesis of rules for intrusion detectors. **IEEE Transactions on Reliability**, IEEE, v. 71, n. 1, p. 370–381, 2021.
- BANERJEE, S.; LAVIE, A. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *In: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. [S.l.: s.n.], 2005. p. 65–72.
- BUSINESS, V. **2018 Data Breach Investigations Report**. New York, USA, 2018. Disponível em: https://www.verizon.com/business/resources/reports/DBIR_2018_Report.pdf.
- BUSINESS, V. **2025 Data Breach Investigations Report (DBIR)**. USA, 2025. C. David Hylander, Philippe Langlois, Alex Pinto, Suzanne Widup (eds.). Disponível em: <https://www.verizon.com/business/resources/T437/reports/2025-dbir-data-breach-investigations-report.pdf>.
- CHOWDHARY, K. R. Natural language processing. *In: _____. Fundamentals of Artificial Intelligence*. New Delhi: Springer India, 2020. p. 603–649. ISBN 978-81-322-3972-7.
- DIMITROV, D.; NIKOLOV, D. Leveraging yara and sigma rules to detect chinese state-sponsored hacking groups of the "typhoon" type. *In: Environment. Technology. Resources. Proceedings of the 16th International Scientific and Practical Conference*. Rezekne, Latvia: RTU Press, 2025. II, p. 107–113. Disponível em: <https://doi.org/10.17770/etr2025vol2.8617>.
- DU, M.; HU, W.; HEWLETT, W. AutoCombo: Automatic malware signature generation through combination rule mining. *In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. [S.l.]: ACM, 2021. p. 3777–3786.
- EXABEAM. **AI SIEM: How SIEM with AI/ML is Revolutionizing the SOC**. 2025. Disponível em: <https://www.exabeam.com/explainers/siem/ai-siem-how-siem-with-ai-ml-is-revolutionizing-the-soc/>. Acesso em: 20 jun. 2025.
- HASANOV, I. *et al.* Application of large language models in cybersecurity: A systematic literature review. **IEEE Access**, IEEE, v. 12, p. 176751–176778, 2024.
- JORDAN, M. I.; MITCHELL, T. M. Machine learning: Trends, perspectives, and prospects. **Science**, American Association for the Advancement of Science, v. 349, n. 6245, p. 255–260, 2015.
- Kaspersky. **O que é Jailbreak?** 2024. <https://www.kaspersky.com.br/resource-center/definitions/what-is-jailbreaking>. Accessed: 2025-06-25.
- LANGCHAIN, INC. **Introduction**. 2025. Disponível em: <https://python.langchain.com/docs/introduction>. Acesso em: 20 jun. 2025.
- LEMPINEN, M.; PYYNY, E.; JUNTUNEN, A. **Chatbot for Assessing System Security with OpenAI GPT-3.5**. 2023. 34 p.
- LI, J. *et al.* **GRIDAI: Generating and Repairing Intrusion Detection Rules via Collaboration among Multiple LLM-based Agents**. 2025.
- LIDDY, E. D. Natural language processing. *In: Encyclopedia of Library and Information Science*. 2nd. ed. New York: Marcel Decker, Inc., 2001, (Book Chapter). Disponível via SURFACE, Syracuse University. Disponível em: <https://surface.syr.edu/istpub/63/>.

LIN, C.-Y. ROUGE: A package for automatic evaluation of summaries. *In: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. [S.l.: s.n.], 2004. p. 74–81.

MCGOWAN, M. **Detecting malicious activities with Sigma rules**. 2025. Disponível em: https://lantern.splunk.com/Security/UCE/Guided_Insights/Threat_hunting/Detecting_malicious_activities_with_Sigma_rules. Acesso em: 20 jun. 2025.

MOTLAGH, M. A.; JAFARI, M.; DEHGHANTANHA, A. Large language models in cybersecurity: State-of-the-art. **arXiv preprint arXiv:2401.01910**, 2024. Disponível em: <https://arxiv.org/abs/2401.01910>.

National Institute of Standards and Technology (NIST). **CVE Program: The Process**. 2025. <https://nvd.nist.gov/general/cve-process>. Accessed: 26 June 2025.

National Institute of Standards and Technology (NIST). **National Vulnerability Database (NVD): General**. 2025. <https://nvd.nist.gov/general>. Accessed: 26 June 2025.

National Institute of Standards and Technology (NIST). **National Vulnerability Database (NVD): Vulnerability Metrics**. 2025. <https://nvd.nist.gov/vuln-metrics>. Accessed: 26 June 2025.

NIST. **NVD CWE Slice**. 2024. Acessado em: 26 jun. 2025. Disponível em: <https://nvd.nist.gov/vuln/categories>.

NURUSHEVA, A. *et al.* Machine learning algorithms in siem systems for enhanced detection and management of security events. *In: Bulletin of L. N. Gumilyov Eurasian National University. Mathematics, Computer Science, Mechanics Series*. Astana, Kazakhstan: L. N. Gumilyov Eurasian National University, 2024. v. 148, n. 3, p. 6–17.

OPENAI. **Text generation guide**. 2024. <https://platform.openai.com/docs/guides/text?api-mode=responses>. Acesso em: 22 jun. 2025.

OWASP Foundation. **LLM01: Prompt Injection - OWASP Top 10 for Large Language Model Applications**. 2024. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. Accessed: 2025-06-25.

PALO ALTO NETWORKS. **What Is the Role of AI and ML in Modern SIEM Solutions?** 2025. Disponível em: <https://www.paloaltonetworks.com/cyberpedia/role-of-artificial-intelligence-ai-and-machine-learning-ml-in-siem>. Acesso em: 20 jun. 2025.

PAPINENI, K. *et al.* BLEU: a method for automatic evaluation of machine translation. *In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2002. p. 311–318.

PICUS SECURITY. **What Is a YARA Rule?** 2025. Disponível em: <https://www.picussecurity.com/resource/glossary/what-is-a-yara-rule>. Acesso em: 20 jun. 2025.

PODZINS, O.; ROMANOV, A. Why siem is irreplaceable in a secure it environment? *In: 2019 Open Conference of Electrical, Electronic and Information Sciences (eStream)*. [S.l.: s.n.], 2019. p. 1–5.

ROTH, F. **Rule Creation Guide**. 2022. Disponível em: <https://github.com/SigmaHQ/sigma/wiki/Rule-Creation-Guide>. Acesso em: 20 jun. 2025.

SADDI, V. R. *et al.* Examine the role of generative ai in enhancing threat intelligence and cyber security measures. *In: Proceedings of the 2024 2nd International Conference on Disruptive Technologies (ICDT)*. India: IEEE, 2024. p. 537–542.

SANTOS, G.; MARTINS, J.; EVANGELISTA, G. Prompt engineering com chatgpt no contexto acadêmico de ihc: uma revisão rápida da literatura. *In: Anais do Simpósio Brasileiro de Fatores Humanos em Sistemas Computacionais (IHC)*. Ribeirão Preto, SP, Brasil: Centro Universitário Estácio de Ribeirão Preto, 2024. Disponível em: <https://sol.sbc.org.br/journals/index.php/ihc/article/view/25054>.

SCHWARTZ, Y. *et al.* LLMCloudHunter: Harnessing LLMs for automated extraction of detection rules from cloud-based CTI. *In: Proceedings of the ACM Web Conference 2025 (WWW '25)*. Sydney, NSW, Australia: ACM, 2025. p. 1922–1941. ISBN 979-8-4007-1274-6/25/04.

SCHWARTZ, Y. *et al.* **LLMCloudHunter: Harnessing LLMs for Automated Extraction of Detection Rules from Cloud-Based CTI**. 2024.

SHAH, A. H. *et al.* Automated log analysis and anomaly detection using machine learning. *In: Fuzzy Systems and Data Mining VIII*. Amsterdam, Netherlands: IOS Press, 2022, (Frontiers in Artificial Intelligence and Applications, v. 358). p. 137–147.

SHEPARDSON, D. **SMarriott CEO apologizes for data breach, unsure if China responsible**. 2019. Disponível em: <https://www.reuters.com/article/us-usa-cyber-congress/marriott-ceo-apologizes-for-data-breach-unsure-if-china-responsible-idUSKCN1QO217/>. Acesso em: 20 jun. 2025.

SIGMAHQ. **About Sigma**. 2025. Disponível em: <https://sigmahq.io/docs/guide/about.html>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma - Generic Signature Format for SIEM Systems**. 2025. Disponível em: <https://github.com/SigmaHQ/sigma/tree/master>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma Rules**. 2025. Disponível em: <https://sigmahq.io/docs/basics/rules.html>. Acesso em: 20 jun. 2025.

The MITRE Corporation. **Common Weakness Enumeration (CWE): About CWE**. 2006–2025. <https://cwe.mitre.org/about/index.html>. Accessed: 26 June 2025.

The MITRE Corporation. **About CVE: History**. 2025. <https://www.cve.org/About/History>. Accessed: 26 June 2025.

The MITRE Corporation. **CWE-352: Cross-Site Request Forgery (CSRF)**. 2025. <https://cwe.mitre.org/data/definitions/352.html>. Accessed: 26 June 2025.

The MITRE Corporation. **CWE Usage Guidance**. 2025. https://cwe.mitre.org/documents/cwe_usage/guidance.html. Accessed: 26 June 2025.

The MITRE Corporation. **MITRE ATT&CK®**. 2025. <https://attack.mitre.org/>. Accessed: 26 June 2025.

WANG, H. *et al.* RulePilot: An LLM-powered agent for security rule generation. *In: Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. Rio de Janeiro, Brazil: ACM, 2026.

Wazuh, Inc. **Wazuh - The Open Source Security Platform**. 2024. <https://wazuh.com/>. Copyright © 2015-2024 Wazuh, Inc.; Accessed: 26 June 2025.

WEBER, M. *et al.* **RedPajama: an Open Dataset for Training Large Language Models**. 2024. Disponível em: <https://arxiv.org/abs/2411.12372>.

GLOSSÁRIO

BeautifulSoup . 20

DeepSeek . 22

F1 . 23

GLM . 24

GPT . 20, 22, 24

LangGraph . 26

LLaMA . 22

MITRE . 22, 25

SigmaHQ . 26, 27

Snort . 23

Splunk . 21, 22

Suricata . 23

zero-shot . 27